

ユニット 7・第10-12週

スレッドと並行性

📅 第10-12週・日程は未定 </> 演習 TD6（解答 + 生成AIチャレンジ） 🎞 アニメーション 5 本

マニュアル M7・ユニットの理論

スレッドと並行性

ユニット 7 の基本の読み物（約 2 時間）。出力はすべて実物で、コースのマシン（x86-64、28 コア）で TD6 のプログラムを実行したものです。

目次

- | | |
|--------------------------------|-------------------------|
| §1 スレッド | §4 ミューテックス、セマフォ、条件変数 |
| §2 競合状態とアトミック性 | §5 デッドロック |
| §3 相互排除：ソフトウェアから
ハードウェアへ | |

§1 スレッド

スレッドはプロセスの中の逐次的な実行です。プロセスのスレッドはアドレス空間全体（コード、グローバルデータ、ヒープ）とディスクリプタを共有し、それぞれが自分の**スタック**、**レジスタ**、スケジューリング情報を持ちます。スレッド間にメモリ保護はありません。あるスレッドのバグが全員のデータを壊すこともあり、SIGSEGV のようなシグナルはプロセス全体を終了させます。Linux では、カーネルは各スレッドをタスクの 1 つとしてスケジュールします（メモリとディスクリプタを共有する `clone` で作られます）。

POSIX API： `pthread_create(&t, attr, f, arg)` は新しいスレッドで `f(arg)` を実行し、`pthread_exit`（または `f` からの `return`）で終了します。`pthread_join` はその終了を待ち、戻り値を受け取り、資源を解放します（誰も待たないなら `pthread_detach`）。これらの関数は `-1` ではなく、`0` かエラー番号を返します。`-pthread` を付けてコンパイルします。ページテーブルをコピーしないので、スレッドの作成はプロセスの作成よりずっと安価です：

実際の出力

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

実際の出力

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5, local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2, local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3, local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6, local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7, local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8, local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

実践 → A7.4 スレッド対プロセス・TD6・1

§2 競合状態とアトミック性

$x = x + 2$ のような C の 1 行は、プロセッサにとっては複数の手順です： x をレジスタに読み、足し、書く。2 つのスレッドが同じ変数にこのような行を実行すると、スケジューラは手順をどのようにでもインターリーブでき、ある順序では一方の書き込みがもう一方を**上書き**します。これが**競合状態**です。

$x = 1$ で $x = x+2 \parallel x = x*3$ の場合、ありうるインターリーブは 20 通りで、9 か 5 になるのは逐次的な 2 通りだけ、残りは更新を 1 つ失います。実際には「悪い」順序はまれで、だからこそこの種のバグは再現しにくいのです。TD6 では、保護なしで 100,000 枚のチケットを売る 8 つのスレッドが、数千枚も売りすぎます。

操作が**アトミック**であるとは、他のスレッドに途中の状態を見せずに、全部実行されるかまったく実行されないかのどちらかであることです。C11 以降、同期なしに別々のスレッドから同じ変数へアクセスし、少なくとも 1 つが書き込みなら、それはデータ競合で、動作は未定義です。`volatile` では解決しません。アトミックな操作には `<stdatomic.h>` か `__atomic_*` を使います。

実際の出力

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

実践 → A7.1 競合状態・TD6・2

§3 相互排除：ソフトウェアからハードウェアへ

クリティカルセクションとは、一度に 1 つのスレッドしか実行できない部分です（**相互排除**）。「牛乳が多すぎる」問題は、読み書きだけでこれを実現するのがいかに難しいかを示します。見てから買うのでは隙間が残り、共有のメモも同じです（見ることと貼ることは 2 つの手順）。スレッドごとのメモは二重に買う

ことは防ぎますが、誰も買わないことがあります。**Peterson のアルゴリズム**（自分のメモ + 順番を譲る + 相手にメモがあり相手の番である間は待つ）は、メモリが逐次一貫なら正しいのです。

実際のプロセッサはそうではありません。x86 では各コアが書き込みを**バッファ**に残し、それが他のコアに見える前にメモリを読むことがあります（TSO モデル）。ARM はさらに並べ替えます。そのため Peterson の 2 つのスレッドが互いのメモを 0 と読み、一緒に入ることがあります。授業用マシンで、メモリフェンス（`mfence`）ありとなしで測った結果：

実際の出力

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

だから相互排除は**ハードウェアのアトミック命令**で作ります：test-and-set、compare-and-swap（x86 では `lock cmpxchg`）、fetch-and-add。ループで待つロック（**スピンロック**）は CPU を浪費し、優先度逆転を起こすこともあります。Linux の解決策は **futex** です：ロックが空いていればユーザ空間のアトミック命令で取り、待つ必要があるときだけカーネルを呼んで眠ります。競合がなければ、ミューテックスはシステムコールをほとんど行いません：

実際の出力

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex
./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00 0.000000 0 1 futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00 0.223730 27 8013 3116 futex
```

実践 → A7.5 牛乳問題・TD6・5

§4 ミューテックス、セマフォ、条件変数

ミューテックス（`pthread_mutex_lock` / `unlock`）はクリティカルセクションを守ります：

ミューテックスが取られていれば、解放されるまで眠ります。**セマフォ**（Dijkstra）は 2 つのアトミック操作を持つ 0 以上の整数です：`sem_wait`（P：正になるまで待って 1 引く）と `sem_post`（V：1 足して待っているスレッドを起こす）。1 で初期化すればロック、0 ならスレッド間の通知、k なら同時に k

人までの制限になります。 `sem_init` は名前なしのセマフォを作り（共有メモリ上で `pshared = 1` ならプロセス間で共有可能）、 `sem_open` は名前付きのものを作ります（失敗すると `SEM_FAILED` を返します）。

生産者-消費者は両方を組み合わせます： `BUF` 個のセルのバッファ、 `empty` セマフォ（空きセル、初期値 `BUF` ）、 `full` セマフォ（埋まったセル、初期値 0）、共有の添字のためのミューテックス。生産者はバッファが満杯なら、消費者は空なら待ちます。何も失われず、重複もしません：

実際の出力

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

任意の条件を待つには**条件変数**を使います：ミューテックスを持った状態で

`while (!condition) pthread_cond_wait(&c, &m);`。 `cond_wait` はミューテックスを放して眠るのをアトミックに行い、起きたときに取り直します。条件を変えた側は `pthread_cond_signal` か `_broadcast` を呼びます。偽の起床がありうるので `while` を使います。

実践 → A7.2 生産者-消費者・TD6・3

§5 デッドロック

デッドロックとは、スレッドの集まりが互いを永遠に待ち続けることです。Coffman の**4つの条件がすべて**成り立つときだけ起きます：相互排除、他の資源を待ちながら資源を保持すること、横取り不可、循環待ち。実行順序次第なので、何千回テストしても現れないことがあります。食事する哲学者がそれを示します：各自がまず右の箸、次に左の箸を取ると、全員が1本ずつ持ったまま隣の人の箸を待つことになりえます。条件を1つ崩せば十分です：取得の**全体順序**（TD6の修正）、競う人数を制限する（「給仕」）、放してやり直す（`trylock`）、あるいは割り当てグラフの循環を検出して回復する。Linux カーネルでは `lockdep` が、ロックが常に同じ順序で取られることを確かめます。

実際の出力

```
$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status: ${PIPESTATUS[0]}"
(124 = still blocked after 3 s)"
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)
```

実際の出力

```
$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals
```

実践 → A7.3 哲学者・TD6・4